

VCF Core: A Semantic Foundation for RDF Representations of VCF

Elias Crum^{1,2,*}, Bart Buelens², Gökhan Ertaylan² and Ruben Taelman¹

¹IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

²Flemish institute for Technological Research (VITO) Mol, Belgium

Abstract

Medical applications of genomic data depend on connecting variant observations to clinical information and biomedical knowledge. Linked Data offers a framework for this integration, but the Variant Call Format (VCF) does not natively provide an RDF representation of the observations and their interpretation context. We present VCF Core Vocabulary 2.1.2, a shared semantic target for representing VCF files, header declarations, records, annotations and sample data. The vocabulary integrates existing provenance and genomic models while retaining VCF-specific field definitions, ordering and missingness. *Expanded* sample calls expose individual values, whereas *condensed*, sample-ordered vectors provide an alternative for cohort data. Version-specific registries and validation profiles address VCF 4.1–4.5, including 122 reserved INFO and FORMAT declarations for VCF 4.5. A curated assessment records representation evidence for all 104 inventoried logical-model constructs and enforcement evidence for 87, and a set of 123 curated alignments documents connections to existing variation models and reference ontologies. We evaluate the vocabulary with executed checks rather than term counts. A specification-derived assessment of 94 *requirements* over VCF 4.1–4.5 supplies *queries* and *expected answers* authored in advance. Re-executing all of it over graphs from a second, separately implemented producer — every *fixture*, every *case* that has a test — gives 840 checks, 693 passing for both producers. The 102 checks where they differ span 15 *requirements*, and examining each shows that none arises from the vocabulary being unable to express something: every absent property is one the release declares and the converter does not yet write. A semantic round-trip recovers the fixed columns of all 109 records and all 151 sample genotypes, in the *expanded* profile, from structured properties alone, and one integration example answers an allele-dependent question under VCF 4.5 local alleles with checked answers and recovered provenance. The independent RDF converter comparison also found a silent correctness defect in local-allele handling that per-producer testing had not, which we report as evidence for keeping representational mechanism, implementation and correctness apart. We then discuss how this foundation could support reusable conversion engines, traceable links to biomedical resources, and SPARQL queries combining genomic observations with medically relevant information.

Keywords

Variant Call Format, RDF, Genomic data, Vocabulary, Data integration, SHACL

1. Introduction

Healthcare research and personalized medicine depend on integrating clinical observations, molecular data, and biomedical knowledge. These resources are distributed across institutions and databases, where differences in data structures, identifiers, and terminology complicate their combined use [1, 2]. Semantic Web approaches provide a basis for addressing this heterogeneity through shared identifiers, explicit relationships and reusable vocabularies [3, 4]. Leveraging these strategies has enabled harmonizing clinical data across institutions while integrating knowledge about genes, diseases, and phenotypes to support biomedical investigations [1, 2]. Meaningful integration of these distributed resources nevertheless depends on retaining the context in which source observations were recorded, not only on adopting common identifiers or terminology [1]. For genomic data, this motivates a reusable semantic representation that preserves source observations and their interpretation context while enabling

SWAT4HCLS 2027, February 1–4, 2027, Basel, Switzerland

*Corresponding author.

✉ elias.crum@ugent.be (E. Crum); bart.buelens@vito.be (B. Buelens); gokhan.ertaylan@vito.be (G. Ertaylan); ruben.taelman@ugent.be (R. Taelman)

🆔 0009-0005-3991-754X (E. Crum); 0000-0001-7734-3747 (B. Buelens); 0000-0001-5602-6435 (G. Ertaylan); 0000-0001-5118-256X (R. Taelman)



© 2027 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

connections to clinical information and biomedical knowledge.

The Variant Call Format (VCF) is widely used to exchange genomic variant data [5]. A VCF file can be understood as a structured table in which each data row describes a genomic location and the sequence observed there relative to a reference genome. For each row, a record contains the reference allele, one or more alternative alleles, and optional annotations describing properties such as quality, filtering, or predicted biological effects. When individual samples are included, additional columns record sample-level observations such as the genotype or sequencing depth. A header at the beginning of the file defines many of these fields and specifies how their values should be interpreted [6].

VCF format has been evolving since its first public release in 2011. At the time of writing, the specifications are maintained by the Global Alliance for Genomics and Health (GA4GH) [7] with versions 4.1–4.5 hosted on a GA4GH-maintained github repository [8]. The most current specification is VCF v4.5, released February 25, 2026 [6]. VCF 4.1–4.5 share the basic structures described above but differ in supported declarations and constraints [9, 6]. For example, later versions introduce new ways to describe variants and their associated metadata, while also refining how existing fields should be interpreted. These relationships are important because VCF often encodes meaning indirectly through the structure of the file. For example, the reference allele is assigned index 0 and alternative alleles are numbered in the order in which they appear. A genotype written as 0/2 therefore does not itself identify two DNA sequences: it indicates that the sample carries the reference allele and the second listed alternative allele. Similarly, the meaning and expected number of values for an annotation may depend on a definition given in the file header. This simple example illustrates a broader characteristic of VCF: the format achieves compactness by relying on relationships that are understood within the file rather than stated explicitly with each value. More complex *cases* depend on several such relationships at once. When VCF observations are integrated with information outside the file, these implicit relationships must remain available if the observations are to be integrated meaningfully with other genomic, clinical, and biomedical data. The challenge is therefore not that VCF lacks contextual information, but that much of this context is encoded through the internal structure of the file rather than exposed as explicit, independently addressable relationships.

A semantic representation offers a way to make these dependencies explicit while preserving their connection to the original observations. Doing so supports semantic interoperability and reuse, central concerns of the Findable, Accessible, Interoperable and Reusable (FAIR) principles [10]. The wider FAIRness of a dataset also depends on its identifiers, metadata, access conditions and provenance, rather than on the choice of file format alone [11]. The Resource Description Framework (RDF) provides a graph-based model for representing resources and their relationships, while Linked Data principles support the use of shared identifiers and links across datasets [12, 13]. Together, these provide a basis for exposing VCF observations and their interpretation context alongside connections to external biomedical resources, which is necessary for any downstream clinical or research applications [14].

1.1. Related Work

Related work includes reusable semantic models and VCF–RDF conversion or query tools, with some contributions combining both roles (Table 1). The Genomic Feature and Variation Ontology (GFVO), Genome Variation Ontology (GVO), and HERO-Genomics model genomic or clinical concepts; tools such as VCF2RDF, TogoVar, BioInterchange, AgroLD and bio-vcf [15] expose VCF-derived data for integration. The GA4GH Variation Representation Specification (VRS) complements these approaches with computable variation descriptions and federated identification, using JSON Schema rather than RDF or OWL; it does not preserve source-file structure [16].

These approaches distinguish source observations from biological interpretations. VCF Core provides a shared, version-aware representation of source observations with links to existing genomic models, allowing converters and queries to reuse VCF interpretation rules.

The VCF Core Vocabulary. Here, we present and assess the VCF Core Vocabulary (*vcfcv*) [29]. It combines source-context preservation, *expanded* and *condensed* sample representations, 123 curated

Table 1

VCF-related approaches and their roles: reusable models, RDF conversion/query tools, or combinations of both.

Approach	Role and focus
BioHackathon <i>vcf2rdf</i> [17]	Converter. Early VCF–RDF conversion, using FALDO for positions and tool-specific terms for other VCF concepts.
GFVO [18]	OWL ontology. Harmonizes genomic features and variation across VCF, GVF, GFF3 and GTF.
FALDO / BioInterchange [19, 18] VCF2RDF [20]	Ontology + converter. BioInterchange reuses FALDO for genomic locations in VCF–RDF conversion; FALDO does not model VCF structure or fields. Mapping/converter. Isomorphic VCF–RDF mapping exposing file components independently with source context; not a general genomic ontology.
TogoVar <i>vcf2rdf</i> [21]	Converter. Simple RDF representation of population allele-frequency VCF data for integration with TogoVar’s biomedical knowledge graphs.
GVO [22]	OWL ontology. Describes biological variation, particularly complex structural variants, rather than VCF structure and interpretation rules.
VariantKG [23]	Converter + application ontology. Builds large VCF-derived graphs for variant analysis and graph machine learning, rather than VCF specification modelling.
SPARQLing genomics [24]	Converter + query platform. Generates RDF from VCF and other genomic sources and serves it through a query interface; its terms are tool-specific rather than a model of the VCF specification.
Semantic Beacons [25]	Integration mapping. Maps GA4GH Beacon responses to RDF with FALDO, SO and GENO for federated genomic querying; not a VCF–RDF representation.
HERO-Genomics [26]	Ontology. Integrates VCF- and Beacon-derived information across clinical and research settings, with broader scope than VCF specification coverage.
AgroLD VCF2RDF [27]	Converter. Incorporates VCF variation into AgroLD’s plant-science knowledge graph; not a dedicated VCF ontology.
VCF-RDFizer [28]	Converter + precursor vocabulary. Our earlier VCF–RDF workflow; VCF Core separates its model from conversion and narrows its scope to the VCF specification.

alignments to existing models, and version-specific artifacts for VCF 4.1–4.5. Four questions structure its assessment: **RQ1:** does VCF Core preserve the intended meaning of the VCF constructs it claims to support? **RQ2:** do the version-specific artifacts and the two *sample profiles* behave as documented? **RQ3:** does the resulting representation support a reproducible integration task while retaining source context? **RQ4:** how does VCF Core relate to other existing VCF semantic models?

Our results answer these questions as follows. **RQ1:** VCF Core is shown to (a) directly conform to roughly half of all extracted VCF specification *requirements* from VCF 4.1–4.5 with gaps coming from untested *requirements* rather than failures, and (b) a structured representation for every construct in a curated inventory of VCF 4.5. Thus, we assert that we cover a large portion of the *requirements* in VCF specifications 4.1–4.5 and all the terms defined in our vocabulary can be mapped back to the original VCF specification constructs. **RQ2:** the version-specific registries reproduce declarations extracted from the specification texts; a second, separately implemented converter answers the same questions the same way on nearly all checks; the two *sample profiles* agree on every question above the sample level and diverge, by design, only below it; and a file’s logical content can be rebuilt almost entirely from the RDF alone, without falling back on any preserved copy of the original text. **RQ3:** an end-to-end allele-dependent integration example returns the *expected answers* with their source provenance. **RQ4:** inspecting four published VCF-related models against nine retrieval criteria shows that VCF Core’s distinguishing role is not broader modelling of genomic variation, but making the interpretation context a VCF file carries about itself directly addressable as graph structure rather than leaving each application to reconstruct it.

These assessments deliberately separate the existence of a representational mechanism from its im-

plementation and from the correctness of the resulting data. We then discuss how this foundation could support reusable conversion engines and integrated, medically-relevant genomic querying. Throughout, `vcfc`: denotes <https://w3id.org/vcf-core/vocab#>.

2. Methods

2.1. Research questions and evidence standard

Our four research questions (RQ1–RQ4) are designed to assess the vocabulary’s ability to preserve the meaning of VCF constructs, to support independent implementations, to integrate with other data sources, and to compare with existing models. Coverage claims are easy to overstate because three different measurements are routinely reported as one number, so we keep them apart: *a requirement exists* when a term, rule or documented pattern is available, *an example instantiates it* when a *fixture* shows it in use, and *behaviour is independently checked* when an executed *query* agrees with an *expected answer* authored from the specification before that *query* is run. Only the third supports a correctness claim, and only within what was tested. All methodology described in RQ1–RQ3 adapt established competency-question, *requirements*-specification, and ontology-testing practices from academic publications cited below. RQ4 asks a different kind of question — whether our *vcfcv* contribution was needed at all — and is answered by inspection rather than experimentation. All procedures, code, and pinned environments are publicly available at https://github.com/ecrum19/SWAT4HCLS_2027, one directory per question, each with an `EXPERIMENT.md` file recording its detailed design, executed steps, stated weaknesses, and results interpretation.

2.2. The representation under assessment

VCF Core was developed iteratively from the earlier VCF-RDFizer vocabulary [28], combining Claude Opus 5 and GPT-6 Astra assistance with expert human review to extract constructs, field meanings, and constraints from VCF 4.1–4.5 specifications into vocabulary terms, version-specific field definitions, and SHACL shapes [9, 6]. The resulting model keeps a file, its header declarations, its records, and its sample observations as distinct resources, with field definitions separate from their occurrence in a header and each value linked to the declaration needed to interpret it (Figure 1); PROV and DCAT are reused for source descriptions, and SO and FALDO connect to alteration and location models without normalizing variants or asserting instance identity across files [30, 31, 19, 29]. VCF Core is deliberately a semantic vocabulary rather than an axiomatized ontology: it declares classes, properties, domains, ranges and a shallow subclass hierarchy with documented definitions, and carries no OWL restrictions, cardinality, disjointness or inverse axioms. Cardinality and value rules are instead expressed as SHACL shapes [32], because a field’s declared `Number` constrains what a conformant file may contain rather than licensing inferences about what a graph must entail. For more information about the vocabulary’s design and contents, an interactive companion application, *VCF Core Vocabulary Explorer*, was created for users to browse the terms, their definitions, and their relationships using the open source tool *Ontology Companion Generator* v1.4.0 [33].

Within VCF Core, two *sample profiles* can encode the same VCF sample column data differently. The *expanded* profile gives each sample’s value for each field its own resource, reachable by graph pattern; the *condensed* profile stores sample-ordered vectors for dramatically reducing the number of triples produced in cohort-scale multi-sample VCF files, its `VCFTextVector` encoding retaining tab-separated values in sample order, so reaching one cell requires decoding. Both retain the same file, header and record context, and VCF 4.1–4.5 share one model, selecting version-specific registries and validation overlays from the file’s declared `fileFormat` [9, 6, 29]. Every RQ1 *case* runs in both profiles; the round-trip and the integration study use the *expanded* profile only, for reasons given below.

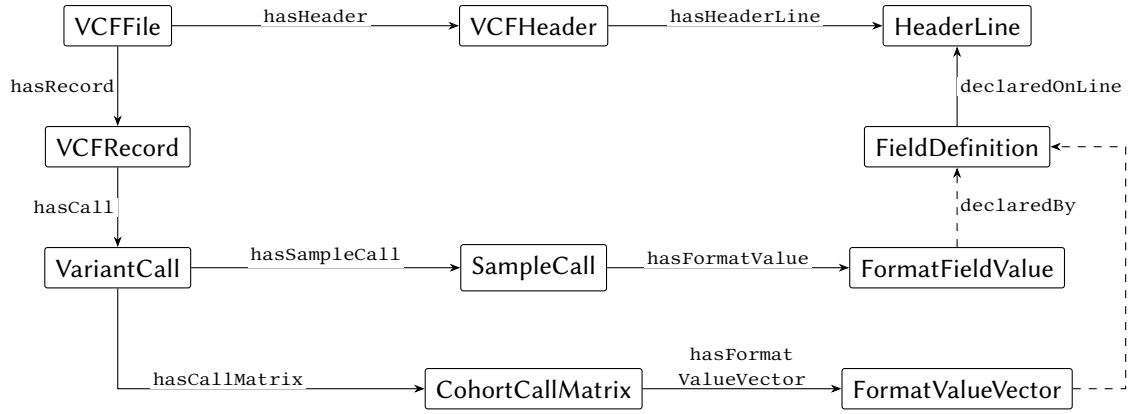


Figure 1: Selected VCF Core relationships that retain implicit VCF format information; prefixes omitted. A VariantCall groups a record’s observations rather than naming an extra source line, and a VCFSample identifies a source column. The two profiles differ only below VariantCall: *expanded* defines a SampleCall and FormatFieldValue per sample and field, *condensed* defines a CohortCallMatrix of sample-ordered vectors. Dashed edges reach the declaration giving a value its type and cardinality.

2.3. RQ1: two assessments, from opposite directions

A semantic vocabulary, intended to represent a logical source, can contain faults in two primary dimensions [34], and one test cannot easily catch both. It can be *missing something* (**completeness**): the logical source (i.e. the VCF specification) defines *requirements* that are not represented in the vocabulary. It can also *define a term not represented in the logical source* (**conciseness**): the vocabulary contains terms that do not correspond to *requirements* dictated by the logical source. To assess both of these fault dimensions, we run two complementary assessments, one going from the specification to the vocabulary [35] and the other going from the vocabulary to the specification. We performed and reported the results of these tests separately.

2.3.1. Specification-derived requirements (specification → vocabulary).

This assessment (RQ1/methodology/) starts from local copies of the VCF 4.1–4.5 B^TE^X specifications, verified against SHA-256 digests; a script partitions each into section intervals and extracts the explicit reserved INFO/FORMAT Number/Type declarations, while interpreting the prose is an authored, reviewed step recorded per passage with its source line range. A *requirement* is a term or rule defined by the specification, expressed as a question with a checkable answer, following established ontology-engineering approaches that use questions to define and test *requirements* [36, 37, 38]. For example, R16 (found in RQ1/methodology/inputs/requirements.json) asks whether values declared with Number=A can be associated with the correct ALT allele. Each *requirement* also records the exact specification passage from which it was derived, identified by its line range and file SHA-256 digest. Because *requirements* are written from the specification, one can name something the vocabulary turns out not to support. Binary encoding, compression, indexing, producer estimation algorithms and general invalid-input rejection were deemed out of scope for this assessment. The design adapts SAMOD, which pairs a question, a *witness* graph, a *query*, and separately authored answers [39].

Terminology. Five terms carry this assessment, and several sound like plain English but are narrower here; *requirement* is defined above and *sample profile* in Section 2.2.

Fixture A small, spec-compliant VCF file designed to make one or more VCF *requirements* observable rather than to resemble real data. *Fixtures* are the only input to graph construction.

Witness The RDF graph materialized from one *fixture* in one profile, against which *queries* run.

Case One *requirement*, at one VCF version, against one *fixture*. For instance, R16 is one *requirement*, but is assessed over five *cases* (5 VCF version over one *fixture*).

Query The SPARQL algebra [40] that attempts to retrieve an answer from a *witness*. One *query* often serves several *cases*.

Expected answer (Referred to as an *oracle*) The rows that a particular *query* should return, authored by a person from the VCF specification and the *fixture* before it is run. This process was performed following established approaches that operationalize ontology *requirements* as executable *queries* with test data and expected results [41, 42].

Queries were drafted with the same AI assistance as the vocabulary and reviewed by a human; the *expected answers* were primarily obtained from examples within the VCF specifications themselves and implemented by the authors. Answers are compared as multisets after sorting both sides, so an extra, missing or duplicated row fails while row order is not — *requirements* that are *about* order project the ordinal as a value instead. All values are compared as strings, so 30 and 30.0 differ, and a *case* is rejected unless its *fixture*'s `##fileformat` line matches the version it claims.

Two retrieval axes. Every *case* is tested twice against the same graph, with the same question, under different rules about what counts as an acceptable route to the answer. A *preservation query* may do anything, including pulling a value out of a packed string; a *structure query* may not use REPLACE, SUBSTR, STRBEFORE or STRAFTER, and must follow declared relationships instead, a ban enforced by raising an error. Without the distinction a vocabulary that dumped each VCF line into one literal would score perfectly on *preservation*; it is also the only reason the *condensed* profile scores lower, since that profile has no per-sample resources for a *structure query* to walk.

Two controls that stop a test passing for the wrong reason. The *empty-graph control* rejects a *case* whose *query* returns the *expected answer* against an empty graph, catching answers baked into the *query* text. The *predicate-deletion control* removes, in turn, each vocabulary property the *query* mentions and rejects the *case* unless at least one deletion changes the answer; it stops at the first property whose removal matters, so it proves the *query* depends on *some* vocabulary data, not on every property it names.

Scoring. *Requirements* are scored separately per applicable version, profile and axis, and are *demonstrated* only when every *case* for them passes, *partial* when some pass, *not demonstrated* when *cases* exist and none pass, and *unassessed* when no test has been written. The numerator counts demonstrated *requirements* at equal weight; the denominator retains every applicable *requirement*, including *unassessed ones*, with no fractional credit, so untested *requirements* score zero rather than leaving the denominator and the percentages in Section 3.1 are a floor rather than a ceiling.

Graph construction, and the bias it introduces. RQ1's *witness* graphs are built by the vocabulary repository's own Python *materializer*, which reads a *fixture* as text and emits resources for the file, header, records, alleles, samples, genotypes and field values. It is not a converter and does not claim to be; its relevant property is that it fails rather than guesses so incomplete graphs are not likely. The materializer, the vocabulary, and the *queries* share authors, so a shape agreed between materializer and *query* could pass whether or not it reflects what VCF means. Four features of the design bound that risk: a wrong *value* still fails, because the answers come from the specification; the materializer emits 200 distinct vocabulary terms against the 79 *queries*' 125 references, leaving 81 emitted terms no *query* asks for; the two controls reject tests that pass without data; and RQ2 puts the same questions to a separately implemented producer.

2.3.2. Curated construct inventory (vocabulary → specification).

The second assessment evaluates the vocabulary against a curated inventory of 104 logical constructs defined in the VCF 4.5 specification (RQ1/vcf45-inventory/). Each construct in RQ1/vcf45-inventory/inventory.json is grouped by specification area and linked to the vocabulary terms and

evidence intended to represent it. This follows established ontology-evaluation approaches in which a vocabulary is assessed against an external reference [34, 35]. For each construct, reviewers recorded three separate judgments: whether the relevant information was *preserved*, whether it was *structured* so that it could be retrieved through graph patterns without parsing a literal value, and whether a validation rule *enforced* the corresponding specification *requirement*. Representation was classified as *full* when both preservation and structure were demonstrated, *partial* when the information was preserved but not structurally represented, and *absent* otherwise. Enforcement was assessed separately and did not contribute to the representation score.

2.4. RQ2: assessment over graphs generated by an independent RDF converter

RQ2 asks whether two documented claims hold: that the version-specific artifacts behave as documented, and that the two *sample profiles* do. Each is tested by different means. The first is a comparison of tables. Every VCF version predefines a set of INFO and FORMAT fields together with the Number and Type each must carry, and VCF Core records these in a *registry* for each version. We extract those declarations from the specification texts and check them against the matching registry. This compares the vocabulary directly against the specification, so its results are reported alongside RQ1's.

The second claim is tested by an *independent RDF converter comparison*: RQ1's assessment run a second time with exactly one thing changed. The *fixtures*, the *queries*, and the *expected answers* are RQ1's, reused unchanged; only the program that turns a VCF file into RDF is different. Any difference in the outcome is therefore attributable to that program, and not to a difference in what was asked. The second program is VCF-RDFizer 3.0.3 (be658a2), run from its published container image pinned by digest sha256:31f1361b; it builds graphs by executing RML mappings with RMLStreamer, rather than constructing triples in Python as RQ1's materializer does [43]. Where the two programs agree, a claim about the vocabulary rests on two implementations instead of one; where they disagree, at least one of them is wrong, and the useful question is which.

The programs share an author, but the tool's RML-based conversion strategy is completely independent of RQ1-related Python conversions. Thus, the comparison is designed to catch implementation mistakes and is not presented as independent replication by an unrelated group. Crucially, RQ2 does not make any changes to *queries* or *oracles*.

Scope and scoring. All *fixtures* are converted in both profiles, so every *case* that has a test is reassessed on the graph generated from the *independent RDF converter* and all *requirements* the *cases* exercise are compared; the *fixture* set is derived at run time from the *case* register rather than listed in the script. Each *case*, axis, and profile yields one of four outcomes — both pass, materializer passes and converter fails, materializer fails and converter passes, or both fail — and every outcome is recorded, not only the disagreements. The materializer's side is *built during the run* by calling the same function RQ1 uses; the committed *witness* files serve as a guard rather than an input, and the run aborts naming the file if a rebuilt graph differs from its record. One comparison rule is a real weakness: because the two producers legitimately mint resource IRIs under different bases, an IRI with a fragment is compared by its fragment alone, so a producer emitting the right fragment under a wrong namespace would pass. Since every *case* already runs in both profiles, the documented claim that the profiles are alternatives rather than a full and a reduced version is tested at no extra cost, by comparing their outcomes *case* by *case*.

Semantic round-trip. A complementary check recovers logical content instead of comparing *query* answers: it rebuilds VCF columns from the RDF alone and compares them with the source file. One script (RQ2/analysis/round-trip.py) rebuilds the seven fixed columns (CHROM, POS, ID, REF, ALT, QUAL, FILTER) and every sample's GT, taking REF and ALT from ordered `alleleIndex/alleleValue` resources and genotypes from ordered `GenotypeAlleleCall` resources with their per-call `phaseIndicator`. A second script (RQ2/analysis/field-round-trip.py) does the same for every header line, recovered by its own source line number and compared on

key and verbatim value, and for every INFO entry and non-GT FORMAT subfield, rebuilt from ordered value items. Neither names a field, *fixture*, sample, contig or expected value: what to look for comes from the source file, how to find it from properties the vocabulary declares. Both are forbidden the preserved-source properties `infoRaw`, `sampleDataRaw` and `genotypeString`, since reconstructing from those would prove only that a string had been copied; their absence from all six *queries* was checked rather than assumed. Both were verified falsifiable: flipping one `phaseIndicator`, deleting one allele call, corrupting one INFO item, deleting one FORMAT value, altering one header value and deleting one header line each produce exactly one mismatch. One normalization is applied and named: from VCF 4.4 a leading phase indicator is permitted, so `| 0 | 1` and `0 | 1` describe the same phasing, which the vocabulary treats as lexical — only the preserved source string records whether it was written out, so reproducing the character would mean reading the very string this check avoids. It is therefore removed before comparison, while allele order, missing calls, ploidy and every indicator between a pair must still match exactly. The round-trip runs on *expanded*-profile graphs only, because decoding a *condensed* vector needs an accessor the converter does not yet emit (R21).

No sampling, thresholds or tolerances are used in RQ1 or RQ2; comparison is exact equality throughout. A separate script (RQ2/analysis/profile-figures.py) checks that every vocabulary term the converter emits is declared in the cited release, resolved against the *git tag* rather than the working tree, so an uncommitted edit cannot flatter the result.

2.5. RQ3: an integration case study under local alleles

RQ1 and RQ2 stay inside the VCF world, and neither shows the representation is *useful* — that someone can join it to statements produced elsewhere and get a trustworthy answer. RQ3 is one small end-to-end case study, deliberately not a realistic clinical workflow: the more moving parts a demonstration has, the harder it is to say which one produced the answer.

Set-up. The *fixture* `local-alleles-v4.5.vcf` was hand-crafted in the VCF Core Vocabulary repository and converted in the *expanded* profile, resulting in roughly 1,400 triples describing one file with four sites written twice — once with local alleles and once globally — one sample and its genotypes. The *expanded* profile is used alone because the *query* walks per-sample value resources the *condensed* profile does not materialize. The converted graph and a hand-crafted local *annotation snapshot* (RQ3/inputs/annotations.ttl) are then loaded into one in-memory graph: two independently produced sets of statements, sharing no identifiers, placed side by side so a *query* can attempt to integrate them. The *annotation snapshot* holds five flagged alterations, invented and labelled as such in the file, and held locally rather than fetched from a live service so the example returns a deterministic answer.

The Query. The *annotation snapshot* names an alteration by four literals — contig, position, reference allele, alternate allele — and the *query* matches those against the graph (Listing 1). No shared identifier is minted in advance, so the join rule is visible in the *query* text rather than hidden inside an IRI someone generated earlier; a pre-minted link would assume the answer. Three assumptions are made prior to the *query*: that both sides describe the same reference assembly, unchecked against the VCF's `##reference` line; that an alteration is its as-written REF/ALT pair at a position, with no normalization or VRS identifier, so the join would not survive a differently written representation of the same change; and that `sample` is a VCF sample column, not a patient.

Listing 1: The integration *query*'s graph pattern, abridged: the annotation join on four literals, the declaration supplying the cardinality, and the depth reached through `forAllele`, which resolves the `Number=LR` frame of reference so the *query* never handles the LAA indirection itself. Omitted: prefixes, the projection, the file/record/sample-call path, and the `FILTER` on depth.

```
?annotation a ex:FlaggedAlteration ; ex:contig ?contig ; ex:position ?position ;
```

```

ex:referenceAllele ?ref ; ex:alternateAllele ?alt .
?record vcfc:chrom ?contig ; vcfc:pos ?position ; vcfc:ref ?ref ;
vcfc:hasAltAllele ?altAllele .
?altAllele a vcfc:AltAllele ; vcfc:alleleValue ?alt .
?field vcfc:declaredBy ?declaration ; vcfc:hasValueItem ?item .
?declaration vcfc:fieldId "LAD" ; vcfc:fieldNumber ?number .
?item vcfc:forAllele ?altAllele ; vcfc:itemValue ?depth .

```

Pass conditions and thresholds. Three conditions must all hold. The observed rows must equal the expected rows — authored from the *fixture* text before the *query* was run — across all eight projected fields, and not by count, because a positional misreading of Number=LR returns the same number of rows. Each of the three flagged-but-excluded alterations must be confirmed absent *individually*, so absence is for the stated reason rather than because the *query* returned nothing. And every returned row must cite a source file whose IRI names the *fixture* actually converted, all rows citing the same one; the *fixture* name comes from the path the check was given, so this anchor sits outside the graph and cannot be met by internal consistency. The settings in force are a threshold of LAD ≥ 20 , chosen so that the correct reading and a positional misreading select *different* alterations rather than as a clinical threshold, and restrictions to the single Number=LR field LAD and to the five flagged alterations; depth is compared as text, so "30" and "30.0" would differ.

2.6. RQ4: comparison to other VCF semantic models

RQ1–RQ3 test VCF Core against the VCF specification, and none of them says whether it was worth building: if an existing model already did this, the contribution would be redundant. RQ4 therefore addresses: for each of nine questions a user might ask of a VCF graph, can the model’s published artifact answer it, and on what evidence?

Criteria and verdicts. The nine criteria (Table 4) are retrieval questions phrased so that inspecting an artifact provides an answer. Each cell receives one of three verdicts: **E**, explicitly represented, where the artifact provides a documented representation meeting the criterion; **C**, representable with additional conventions the model does not itself establish; **S**, outside the model’s stated scope. **S is a first-class verdict and not a defect:** a model built to type genomic variation is not deficient for lacking header terms, and the scale exists to separate *cannot* from *was never trying to*.

Assessment made through author digression. Unlike RQ1–RQ3, RQ4 is performed through the inspection of the four published artifacts. A script RQ4/*fetch.sh* retrieves all four artifacts locally and records a SHA-256 for each. The four inspected models each state a *different* purpose, which is what makes the contrast informative: GFVO (*gfvo.xml*), HERO-Genomics (*hero_genomics.ttl*), GVO (its published ontology, *owl:versionInfo* 2021-11-18) and VCF2RDF (its 58 published term pages under RQ4/*artifacts/vcf2rdf-terms.tsv*, each resolving to a browsable page) [18, 26, 22, 20]. GA4GH VRS is deliberately not scored: it describes computable variation identity rather than a source file, so scoring it against file-format criteria would manufacture a contrast its authors never sought [16]. Only published artifacts were considered and every cell was settled by inspection.

2.7. Evaluated artifacts and reproduction

All results use VCF Core 2.1.2 [29] and, where a converter is involved, VCF-RDFizer 3.0.3 [43]; both are tagged, so every figure comes from a released artifact rather than a working branch. Each question runs on its own — RQ1/*run.sh*, RQ2/*run.sh*, RQ3/*run.sh* — cloning what it evaluates at a pinned tag into a shared *.artifacts/* directory and pulling the converter image by digest. SWAT4HCL S_2027/PINS.md lists every pinned artifact and digest. RQ1 deliberately runs the scripts *inside* the clone rather than the copies committed here. Each run writes a self-contained directory holding

the machine description and toolchain versions, every command with its exit code and duration, the per-step output, the converted graphs, the result files and a summary; recorded provenance includes the pinned specification revisions with their digests, the *fixture* set, the SPARQL engine used for every *query* (rdflib 7.6.0) and the container image digests.

3. Results

3.1. RQ1: does VCF Core preserve the meaning of the constructs in the VCF specification?

Version-specific declarations. The version registries hold 40, 40, 67, 78 and 122 explicit reserved declarations respectively for VCF 4.1–4.5; the 122 for VCF 4.5 comprise 51 INFO and 71 FORMAT definitions, three of them parameterized base-modification key families. All 333 reserved Number/Type rows extracted from the specification texts (31, 31, 69, 79 and 123 per version) match those registries, with no missing or differing row. Those 333 are the keys each source declares explicitly in `##INFO=`/`##FORMAT=` blocks or reserved-key tables, a different population from the registry entries: VCF 4.1 and 4.2 each define a further 29 reserved keys in prose bullets giving no Number and, for INFO, no Type, which are inventoried but not comparable, so the agreement figure is weighted towards VCF 4.3–4.5. Matching declarations does not resolve inconsistencies between a declaration and its prose meaning: review found CNL/CNP declared `Number=G` alongside prose defining an ordering by copy number.

VCF specification source audit and review. The assessment proceeds in stages. We first separated the VCF specification latex source files into 344 source-section intervals, from which 1,134 passage-level assertions were authored. An assertion becomes a *requirement* only once it can be stated as a question with a variable answer, which yields 94 *requirements*. A *requirement* becomes a *case* when paired with one version and one *fixture*, giving 210 total *cases*. Each *case* includes a *query* and an *expected answer*. Across the 210 *cases*, we defined 79 distinct *queries*, since one *query* often serves several *cases*.

Each of the 1,134 assertions is then classified by whether an executable test reaches it, using two authored fields: the *cases* that test it, and a *test-gap* note recording what those *cases* leave untested. An assertion with at least one *case* and no recorded gap is *targeted* (149), one with *cases* but a recorded gap is *partial* (154), and one with no *cases* at all is *untested* (831). The distinction is visible in three real entries. “Contig ID and supplied length belong to the declared reference sequence” has one *case* for R05 and no gap, so it is targeted. “POS is 1-based and records retain distinct identity and source order even at equal coordinates” has *cases* for R07 and R08 but the authored gap “coincident records need a targeted identity/order *case*”, so its first half is tested and its second is not. “The assembly URL identifies the external sequence source referenced by BKPTID values” has no *case* at all. The links are checked rather than trusted: a cited *case* must exist, share the assertion’s version, name one of its *requirements*, and carry a *query* with an *expected answer*.

These counts are not a coverage score, they only record whether a test exists, not whether it passed, and they are counted once per version, so an assertion stated identically in all five specifications is recorded five times. The 1,134 assertions hold only 385 distinct statements, 86 of them verbatim in every version. Their purpose is to show where audit work has and has not reached; coverage is scored against the 94 *requirements* instead. Which assertions became *requirements* was not arbitrary. An *expected answer* has to be defensible which is only practical where the specification is concrete enough to define an assertion and give an example. Therefore, the investigation primarily concentrated on sections of the specification that provide examples: nine of the 38 *fixtures* are byte-identical copies of published specification examples, and the remainder are minimal files written to instantiate a stated rule. Assertions whose sources gave no defensible serialization and expected-answer basis remained unassessed. Of the 94 *requirements*, along the axis of “intent,” 56 carry an interpretation specific enough to check the *query* against a stated intent, 29 carry a generic intent, and 9 carry an explicit unassessed marker.

Deriving *requirements* from the specification text also exposed defects in those examples, three of which were reported to the VCF specification maintainers via a github issue ([hts-specs#869](#), [hts-specs#870](#), and [hts-specs#871](#)) and one issue already existed ([hts-specs#868](#)). All issues remain open at the time of writing. Each of these complicated authoring the affected *queries* and *expected answers* because the example that would ordinarily inform the *oracle* was itself inconsistent with the assertion. The affected *requirements* were nevertheless scored rather than set aside. Where it was not determinable because the passage is itself disputed upstream, no test was written and the *requirement* was left unassessed, which counts against the score rather than being excluded from it. The reported percentages are therefore depressed by unresolved specification inconsistencies rather than by a gap in the vocabulary.

Specification-derived requirements (specification → vocabulary). The 210 *cases* exercise 53 of the 94 registered *requirements* in at least one applicable version, meaning that these 53 have defined tests. The remaining 41 have no registered *case* in any version, meaning they do not yet have defined tests. Coverage is reported per version, because *requirements* apply per version (Table 2). All 210 *preservation* tests passed in each profile, as did all 210 *expanded-structure* tests. In the *condensed* profile 165 *structure* tests passed and 45 failed, every failure concerns per-sample FORMAT access that the profile stores in encoded vectors rather than individual sample resources. In VCF 4.5, 18 failed *condensed-structure* tests affected 14 *requirements*, failed tests and affected *requirements* are different counts. Every passing *case* rejected the empty-graph control and changed its answer under at least one referenced-predicate deletion. The structural difference is therefore a sample-profile limitation in *condensed* representations, since the corresponding *expanded* tests passed it is not evidence that VCF Core cannot express those values structurally.

Table 2

Demonstrated coverage of registered information *requirements* by VCF version, *sample profile*, and retrieval axis. N is the total number of *requirements* applicable to a given VCF version. ND (*not demonstrated*) counts *requirements* that were tested but did not pass, while U (*unassessed*) counts *requirements* for which no test was available; unassessed *requirements* remain in N and therefore contribute zero to the reported percentage. No *requirement* received a partial result. **Each Demonstrated column is read on its own:** within a column, demonstrated + ND + U = N . For VCF 4.1 that is $32 + 0 + 37 = 69$ in the three agreeing columns and $26 + 6 + 37 = 69$ under *condensed structure*. Row totals are not meaningful, because the four columns report the same *requirements* assessed four ways rather than four disjoint groups. [†]ND is nonzero only for the column immediately to its left, *structure* under *condensed*, and is zero in the other three combinations.

VCF	N	Demonstrated				ND [†]	U
		<i>preservation</i>		<i>structure</i>			
		<i>expanded</i>	<i>condensed</i>	<i>expanded</i>	<i>condensed</i>		
4.1	69	32 (46.4%)	32 (46.4%)	32 (46.4%)	26 (37.7%)	6	37
4.2	72	34 (47.2%)	34 (47.2%)	34 (47.2%)	28 (38.9%)	6	38
4.3	74	35 (47.3%)	35 (47.3%)	35 (47.3%)	29 (39.2%)	6	39
4.4	83	38 (45.8%)	38 (45.8%)	38 (45.8%)	31 (37.3%)	7	45
4.5	91	51 (56.0%)	51 (56.0%)	51 (56.0%)	37 (40.7%)	14	40

How to read these numbers. Across versions, roughly half of the *requirements* are demonstrated for the *expanded* profile and about one third for the *condensed* profile. Importantly, most remaining *requirements* are *untested rather than failed*: the not-demonstrated count is zero for every version, profile, and axis except *condensed structure*. The appropriate interpretation is therefore that the reported percentages show what has been demonstrated, while the remainder is still unassessed. For VCF 4.5, 40 *requirements* remain unassessed. Of these, 37 await planned *fixtures* and *queries*, R45 needs further analysis because interpretation was unclear from the specification wording, and R67–R68 cover all reserved INFO and FORMAT keys and therefore cannot be satisfied by testing only selected *fixtures*.

Curated inventory (vocabulary → specification). All **104** inventoried VCF 4.5 logical constructs were classified as both *preserved* and *structured*, with no partial or absent representations, in every specification area; **87** additionally had a rule checking a stated *requirement*, enforcement being lower for file and header structure (17/20), lexical and encoding rules (6/7), FORMAT and genotype (16/21) and structural variation, repeats and gVCF (8/16) [29]. The evidence behind those judgements was a *query* for 7 constructs, a regression probe for 23, a shape or decoded check for 62 and a materialized *fixture* for 12 — evidence tiers behind curated judgements, not 104 independently executed retrieval tests. Separately, all 17 *fixtures* checked on the bytes pass, on their own axis.

All **104** inventoried VCF 4.5 logical constructs were classified as both *preserved* and *structured*, with no partial or absent representations across any specification area. Of these, **87** also had a validation rule enforcing a stated *requirement*. Enforcement was less complete for file and header structure (17/20), lexical and encoding rules (6/7), FORMAT and genotype (16/21), and structural variation, repeats, and gVCF (8/16) [29]. These classifications were supported by different forms of evidence: 7 constructs by *queries*, 23 by regression probes, 62 by shapes or decoded checks, and 12 by materialized *fixtures*. They should therefore be read as evidence supporting the curated inventory, rather than as 104 independently executed retrieval tests.

3.2. RQ2: do RDF representations of VCF behave as documented?

The same questions, a second producer. RQ2 replicates RQ1 *queries* and *expected answer* assessments on *witnesses* produced by VCF-RDFizer and compares the results to those produced by the VCF Core materializer. Out of 840 *cases* tested, 693 pass for both producers and 45 fail for both; 53 *requirements* are exercised, 38 of them agreeing on every check and 32 passing on every check. The 45 both-fail checks are all *condensed*-profile structure *queries*: both producers answer the same way, and that way is “no” — the documented storage trade-off, not a disagreement. On 102 checks, spanning 15 *requirements*, the materializer answers and the converter does not, and no check shows the inverse. Because the paper’s claim concerns the vocabulary rather than one program, we examined each of the 15 by running the failing *query* against both graphs and checking which properties the converter emits (Table 3).

Table 3

Why the converter fails a check the materializer passes. Every divergent *requirement* is accounted for; the right-hand column is what would have to change to close it.

Cause	Reqs.	What is missing
A declared term the converter does not emit	12	The <i>query</i> asks for a property the vocabulary defines and the converter never writes, so nothing matches. Twenty such properties, including gVCF reference blocks, breakend mates, allele indices, etc.
A decomposition that stops early	1	Tandem repeats: the summary row is emitted, the per-allele components are not.
A datatype the vocabulary leaves open	1	QUAL. The materializer writes <code>vcfc:VCFFloat</code> , the converter <code>xsd:decimal</code> ; both satisfy the shape, and only a <i>query</i> projecting <code>DATATYPE()</code> can tell them apart.
An encoding convention	1	An empty local-allele list, written "" by one producer and ". " by the other.

None of the 15 divergences is the vocabulary being unable to express something. Across all 840 checks, none traces to a representation gap: every property involved is declared in the pinned vocabulary release, including the twenty the converter never writes. What the divergences locate is unfinished or incorrect implementation — 12 of the 15 are declared terms the converter does not yet emit — so they bound what can be said about this converter, not about the model.

That makes the comparison a development instrument as much as a validation one, because it sees what a producer’s own tests cannot. A test suite compares a producer against itself, so it cannot detect

a value that is well-formed, plausible, and attached to the wrong thing. An earlier independent RDF converter comparison found exactly that: the converter bound `Number=LA` and `Number=LR` items to allele indices positionally, as if the list indexed the record's ALT column, when it indexes the sample's local allele subset. Under `LAA=2, 4` a depth list of 20, 30, 10 belongs to global alleles 0, 2 and 4; the converter attached it to 0, 1 and 2 — wrong values rather than missing ones, with nothing in the graph to signal it. That defect and eight further divergences were repaired in response [43], and the pinned release additionally emits the ID, FILTER, field-index and per-allele phase-indicator accessors it had never written, and recognises the thirty base-modification aliases it had ignored. All local-allele *requirements* now pass for both producers on every *expanded*-profile check; one *condensed*-profile divergence remains, R27, described next.

None of the 15 divergences indicates a limitation of the vocabulary itself. Across all 840 checks, no discrepancies trace to a representation gap. The differences identified above instead arise from the implementation of the independent converter VCF-RDFizer: for 12 of the 15 affected *requirements*, the required terms exist in the vocabulary but were not yet emitted by the converter. These results therefore limit what can be claimed about the converter, rather than what the vocabulary can represent. That makes the comparison performed a development instrument as well as a validation one, because it reveals what a producer's own tests cannot.

Semantic round-tripping. Across all 38 *fixtures*, translated into *witnesses* by both conversion mechanisms, reconstructing a VCF file from structured properties alone recovers all **109 records** on the seven fixed columns and all **151 sample genotypes**; the companion check recovers all **431 header lines**, compared by source line number on key and verbatim value, and **400 of 401** INFO entries and non-GT FORMAT subfields. Information therefore survives conversion into the structured layer and can be read back out of it without touching a preserved source string. The one exception is the LAA field of a record whose local-allele list is empty. In this case, the materializer emits the field resource, its `declaredBy` link and `fieldValue` "", while the converter emits the resource and the link but no value triple, so the reconstructed cell reads `0/0:?:30:0`. An empty LAA is meaningful rather than missing, it says this sample's local alleles are the reference only (genotype 0/0), so what is lost is the distinction between *present and empty* and *present without a value*. This is *requirement* R27, reached here by a different route than the independent materialization took: two checks landing on one defect from opposite directions. It is not a representation gap, since the vocabulary expresses the empty value and the materializer writes it, and it does not propagate, since LAD's value item still carries its `forAllele` link on the same record.

Where the two sample profiles (condensed and expanded) agree. The *sample profiles* return the same outcome on **352 of the 420** case-and-axis pairs. All 68 disagreements fall within 15 *requirements* and share one property: every one reads data held *inside* a sample, either a FORMAT value or a genotype. Questions about the file, its header, a record, or an allele agree on every check in both profiles. This discrepancy is in fact the *condensed* profile working as designed. Because the *condensed* profile stores sample values as vectors rather than one resource per sample and field, a *query* written to walk per-sample resources finds nothing, and recovering the value requires a decoding step the included *queries* refuse. The supported claim is therefore narrower than "the profiles are equivalent": they agree above the sample level and diverge by design below it.

VCF Core version-level checks. Converting the worked example emits 242 *expanded* and 168 *condensed* triples, drawing on 93 and 80 distinct VCF Core terms respectively, and every one of those terms is declared in VCF Core 2.1.2 as resolved from the git tag. Separately, the vocabulary's own recorded validation run accepts 21 *fixtures* and includes a negative control, alongside a regression suite of 101 tests. Because the converter pins no vocabulary version, the relationship between the two is established by the recorded behaviour rather than by a declaration, and it is evidence for the terms these graphs exercise rather than for all 467 declared terms or every historical VCF version.

3.3. RQ3: does the representation support an integration task?

Asking which flagged alterations have supporting read depth of at least 20 returns two rows, identical to the answers authored in advance: chr1:1 G>C at depth 30 and chr1:2 A>T at depth 25, each traced to its file, record, sample and the FORMAT declaration giving LAD its cardinality, with every row naming the *fixture* actually converted. The three flagged alterations absent from the answer are each confirmed absent individually and for a stated reason: chr1:1 G>A and chr1:2 A>C because ALT 1 is not in the sample's LAA subset at those positions, and chr1:3 C>G because ALT 1 is not in LAA=3 there and its positional value would fall below the threshold in any case.

This example importantly highlights that a "shallower" representation (i.e. one that cannot semantically represent file, record, sample, and FORMAT fields) gives a different answer than the VCF Core representation. If Number=LR values are interpreted only by position, the *n*th LAD value is matched to the *n*th ALT allele in the record rather than to the *n*th allele in the sample's LAA subset. Under this interpretation, the *query* still returns two rows, but for the wrong alterations: chr1:1 G>A at depth 30 and chr1:2 A>C at depth 25. The result therefore appears structurally plausible: it has the expected number of rows and retains provenance, but the values are attached to the wrong alleles. The expected-answer set records this positional interpretation separately, using only the *fixture* and specification rule defined before *query* execution. Two of the three exclusions change only when Number=LR is resolved through the sample-specific LAA mapping. The difference therefore comes from how the VCF semantics are represented, not simply from whether the underlying values are present. This comparison illustrates the consequence of the specification rule rather than measuring an error in a particular converter. The example is intentionally limited: the input is synthetic, contains four sites and one sample, and uses invented annotations with no biological or clinical meaning. It demonstrates the importance of local-allele-aware modelling for this *fixture*, not correctness across all possible VCF data.

3.4. RQ4: how does VCF Core relate to other VCF semantic models?

Table 4 records the nine verdicts per artifact; three patterns account for most of it.

HERO-Genomics and *GVO* keep INFO as one literal. Both `hero:vcfInfo` and `gvo:info` are datatype properties carrying the whole INFO column as a single string, and both align to `vcfc:infoRaw`, so the information survives but every INFO-derived question requires re-parsing that string *and* supplying the cardinality rule from outside the graph — which is why F7 is C rather than E for both. *GFVO* made the opposite choice, modelling particular field *meanings* as first-class concepts (`AlleleCount`, `Coverage`, `MappingQuality`), which answers those fields well and leaves the declaration mechanism unaddressed, so F2 is S.

VCF2RDF is the instructive case, and it corrected an earlier verdict of ours. Reading the paper alone suggested a term-free isomorphic mapping; the published artifact shows otherwise, with 58 dereferenceable term pages including `Number`, `Type` and `Description`. But with no allele-index term a consumer must still split the value list and count ALT alleles by re-reading a literal: having the declaration is necessary and, on its own, insufficient.

GVO's column is almost entirely S because the retrieved ontology is primarily organized around classes for types of genomic variation. It declares 48 classes (47 of which represent variation types beneath the root class `gvo:Variation`) and 12 datatype properties, but no object properties. As a result, the ontology does not provide relationships for linking variation records to resources such as alleles or samples. This should not be read as a deficiency. *GVO* is designed to classify genomic variation, so the S entries reflect a difference in modelling purpose rather than missing functionality relative to its intended scope. The only C concerns missing values. A literal-valued property can contain ., but the ontology itself does not define how this should be distinguished from the complete absence of a corresponding triple.

Table 4

Can each model’s published artifact answer the question? **E**: explicitly represented. **C**: representable with additional conventions the model does not itself establish. **S**: outside the model’s stated scope — a difference of purpose, not a defect. No cell remained unsettled.

Can a consumer...	VCF Core	GFVO	HERO	GVO	VCF2RDF
F1 trace a record to its file	E	E	E	S	C
F2 read a declaration’s ID, Number, Type as data	E	S	S	S	E
F3 read the declared VCF version	E	S	S	S	C
F4 address ALT allele n by index	E	C	C	S	C
F5 separate ploidy, allele calls and phasing	E	C	C	S	C
F6 get a (sample, field) value unsplit	E	C	C	S	C
F7 bind a Number=A/R/G item to its allele	E	C	C	S	C
F8 distinguish explicit . from “not stated”	E	S	C	C	C
F9 tell which sample encoding was used	E	S	S	S	S

What this supports, and what it does not. *Supported*: VCF Core is distinguished by its representation of the *interpretive context* carried by a VCF file, not by broader modelling of variation itself — GVO’s 47 variation classes are more extensive in that respect. This includes the declaration that determines a field’s cardinality, the VCF version that determines which rule applies, the allele index needed to resolve genotype values, and the sample column that defines the scope of a value.

In the other inspected ontologies this context is handled in one of three ways, each visible in the matrix. HERO and GVO keep the INFO column as a single literal — `hero:vcfInfo` and `gvo:info` both align to `vcfc:infoRaw` — so the text survives, but every INFO-derived question requires parsing that string again and supplying the cardinality rule from outside the graph. GFVO makes the opposite choice, promoting particular field *meanings* to first-class concepts such as `AlleleCount` and `MappingQuality`, which serves those fields well while leaving the declaration mechanism itself unaddressed. VCF2RDF does publish the declaration, with terms for `Number`, `Type` and `Description`, but offers no allele index, so a consumer must still split the value list and count ALT alleles by re-reading a literal.

Not supported: these results do not show that the other models are incapable of representing such information. A label of “requires additional conventions” refers only to what is explicitly established by the inspected ontology artifacts, not to what its community could extend or implement.

Reuse and cross-model alignment are therefore reported separately. VCF Core directly reuses terms from PROV, DCAT, SO, and FALDO, and provides 123 curated SSSOM mappings [44] to variation models and reference ontologies with different levels of confidence, see <https://github.com/ecrum19/vcf-core-vocabulary/tree/main/mappings>. These mappings form a separate alignment artifact and should not be interpreted using the same denominator as the comparison matrix above [29].

4. Discussion

4.1. Assessing VCF Core’s coverage of VCF meaning

This study assesses whether VCF Core makes the information needed to interpret VCF observations available for reuse in RDF. For this purpose, coverage must consider whether a consumer can recover the intended meaning of an observation, including the declaration, allele ordering, and sample context on which it depends. Preserving a field as text is useful for source recovery, but does not establish that these relationships are available to a graph query [34, 35]. RQ1 therefore assessed both preservation and structured retrieval against specification-derived *requirements*, while RQ2 examined whether those answers could also be obtained from a separately implemented converter. The design of both follows established ontology-engineering practice: *requirements* were specified as questions with checkable answers [36, 37, 38], operationalized as executable *queries* paired with independently authored *expected*

answers [41, 42], and organized after SAMOD’s pairing of a question, a *witness* graph, a *query* and separately authored answers [39]. Together, these assessments connect the vocabulary’s declared capabilities to observable behaviour.

The results support broad representation with incomplete demonstrated coverage. All 104 constructs in the curated VCF 4.5 inventory have a documented representation, and 87 also have validation rules expressed as SHACL shapes [32]. However, the executed assessment reaches only 53 of the 94 registered *requirements* across VCF specification versions. For VCF 4.5 specifically, 51 of 91 applicable *requirements* are demonstrated in the *expanded* profile, giving 56.0% coverage under the stated scoring rules (Table 2). All of RQ1’s *expanded*-profile tests pass, while unassessed *requirements* remain in the denominator. These findings establish that the tested relationships can be represented and retrieved, but the remaining uncovered specification rules require further assessment.

The lower *condensed*-profile scores have a different explanation. Both profiles preserve the tested information, but sample-ordered vectors require decoding before individual values can be queried. Consequently, the profiles agree on all tested questions about files, headers, records, and alleles, while direct sample-level retrieval differs. This distinction matters when choosing a representation: the *expanded* profile supports queries that follow individual sample observations, whereas the *condensed* profile retains those observations in a form requiring an additional access step. Coverage is therefore meaningful only in relation to both the information required and the way an application needs to retrieve it.

4.2. Extending demonstrated coverage

The clearest opportunity to improve the coverage figures is to extend the evidence for unassessed *requirements*. The 41 *requirements* without tests include structural-variant fields and version-dependent interpretation rules, for which a term inventory alone provides limited assurance. For VCF 4.5, 37 of the 40 unassessed *requirements* await planned *fixtures* and *queries*; R45 needs clarification of its interpretation, and R67–R68 require systematic coverage of reserved INFO and FORMAT keys. This gives a concrete development agenda: construct examples that isolate each rule, establish their expected answers from the specification, and test them across the versions in which the rule applies [39, 41]. The same process should extend validation beyond the 87 constructs with enforcement evidence, particularly for structural variation and genotype constraints.

While developing the catalogue of VCF specification *requirements*, we identified flaws in the specification’s provided examples. Deriving executable questions exposed inconsistencies in published examples and in the relationship between declarations and prose, including the CNL/CNP cardinality issue described in Section 3.1. These findings show how vocabulary assessment can contribute to specification maintenance: an executable example makes an ambiguous interpretation concrete enough to discuss and revise. This mirrors experience elsewhere: rigorous, executable specifications of network protocols exposed defects in the standards themselves [45], and a conformance assessment of a genomics file format found that widely used tools disagreed about it [46]. Comparable recommendations have been made for VCF in plant genomics, where reuse was impeded less by missing data than by unstated formatting conventions [47].

The present evidence is also shaped by the availability of specification examples. Rules with clear examples were easier to turn into tests, so the demonstrated subset should not be treated as representative of all VCF complexity. Additional allele-dependent examples, difficult boundary cases, and files from different production pipelines are needed to determine whether the same mechanisms remain sufficient beyond the current fixtures.

4.3. A reusable assessment for conversion engines

RQ2 shows that the assessment can guide converter development as well as evaluate the vocabulary. Reusing the same *fixtures*, *queries* and *expected answers* across two implementations produced 693 checks passing for both and 102 disagreements across 15 *requirements*. The remaining 45 checks failed

for both because they requested sample-level structure absent from the *condensed* profile. Examining the disagreements identified specific implementation choices and omissions: twelve affected *requirements* depend on declared properties the converter does not emit, while the others concern incomplete tandem-repeat decomposition, a permitted datatype difference and an empty-value encoding convention. None of the observed disagreements required extending the vocabulary to express the requested information.

More significantly, an earlier RQ2 assessment detected a local-allele error that the converter’s existing tests had missed. For LAA=2, 4, the converter associated three depth values with global alleles 0, 1 and 2 instead of 0, 2 and 4. The values remained well formed, but were attached to the wrong alleles. Testing against answers established from the specification exposed this error and led to its correction. Similarly, our semantic round-trip testing provides a complementary check, recovering all tested fixed columns and genotypes, along with 400 of 401 INFO and non-GT FORMAT fields. Its one mismatch exposed a lost distinction between an empty local-allele list and a field without a value. These findings demonstrate why value recovery and allele association should accompany validation of graph structure.

A shared vocabulary makes these checks reusable across conversion strategies. Implementers can develop streaming, mapping-based, or materializing converters [20, 21, 27, 15] against the same representation and evaluate their outputs with the same questions. Users can then assess a converter against the fields and sample profile their application requires before comparing runtime, memory use or graph size.

4.4. Integrating observations with external knowledge

RQ3 demonstrates an example of the practical value of representing VCF information using a semantic model like VCF Core – when observations are joined to statements produced outside of the file. We chose local alleles because they make the dependence on interpretation context particularly clear. A record can list alternate alleles observed across a cohort, while an individual sample’s LAA identifies a subset of those alleles. Values in LAD, declared with Number=LR, refer to the reference allele and that sample-specific subset. Retrieving depth for an externally annotated alteration therefore requires the field declaration, the sample’s local mapping, and the corresponding record allele. The VCF specification recommends abstracting local-allele encoding within libraries [6]; VCF Core makes the resolved relationship available to graph consumers through `forAllele`.

The example returned the two expected alterations with their supporting depths and source context. A positional interpretation also returned two rows with provenance, but attached those depths to different alterations. Thus, neither the result count nor the availability of provenance was sufficient to establish a correct answer. The useful contribution is that a query can follow an explicit allele relationship while combining source observations with external annotations, without reimplementing the local-allele decoding rule. This creates a practical point of reuse: the interpretation can be implemented and checked during conversion, then used by multiple downstream queries.

These preliminary results are encouraging because the example demonstrates a case in which the structure of the VCF representation directly affects which alterations satisfy the query. It therefore illustrates a useful capability of the vocabulary: relationships that are implicit in the source file can be made explicit and queried together. At the same time, this is only a small proof-of-concept. The input contains four synthetic sites and one sample, and the annotations carry no biological or clinical meaning. The join also assumes a shared reference assembly and exact agreement in the written coordinates and alleles. The example therefore demonstrates a useful property of the representation, not a mature clinical use case or an alternative to existing variant interpretation workflows.

4.5. VCF Core among existing models

The comparison in RQ4 clarifies where this contribution fits within existing genomic models. For the inspected ontologies (see table 4), all exhibit limitations that complicate their use for VCF file data representation on common VCF usecases [18, 26, 22, 20]. VCF Core brings these concerns together: declarations, version information, ordered alleles, sample values, and missingness are represented

within a shared model of source observations. For the tested *expanded* representations, this context can be retrieved through graph relationships.

This role is complementary to models of biological variation and clinical information. For example, observations from two laboratories could retain their own quality values, sample context, and provenance while linking, after normalization, to a common variation description such as one defined using GA4GH VRS [16]. Similarly, a clinical model such as HERO-Genomics [26] could connect VCF sample columns to specimens and patients without treating a source column identifier as a patient identifier. Neighbouring representations such as Med2RDF and the *gvar* schema describe overlapping subject matter and are candidates for the same treatment [48, 49]. VCF Core’s reuse of PROV, DCAT, SO and FALDO [30, 31, 19], together with its 123 curated alignments, recorded in SSSOM and expressed with SKOS mapping predicates [44, 50], provides starting points for these connections. Those alignments are deliberately graded rather than uniform — 57 `skos:exactMatch`, 40 `skos:closeMatch`, 16 `skos:relatedMatch` and 10 `skos:narrowMatch` — so fewer than half assert equivalence, and the set locates related terms in other vocabularies rather than establishing identity between instances.

The comparison remains specific to source interpretation. Its nine criteria reflect the problems VCF Core was designed to address, and the verdicts were assigned through ontology inspection, not direct usage. Review by those models’ maintainers would strengthen both the comparison and the proposed connections between them, following the community-review practice established in ontology consortia [3].

4.6. Future work and independent evaluation

Future integration studies should test whether preserving this context improves the reliability and reuse of queries over real genomic and biomedical data. Relevant scenarios include tracing a pharmacogenomic interpretation to its supporting observations [51, 52], combining rare-disease candidates with phenotype and gene–disease information [53, 2], and linking tumour observations to curated therapeutic evidence [54]. These remain proposed research workflows. Each requires additional biomedical models, independently reviewed mappings, and explicit relationships between samples, specimens and patients. The contribution to evaluate is whether VCF Core makes the source evidence easier to retrieve and verify within those workflows.

Practical evaluation should also compare the two sample profiles on realistic cohort data, measuring conversion cost, storage, loading, and query execution alongside answer correctness. The current examples do not establish performance at scale, and the round-trip and integration results apply only to the *expanded* profile. Investigating how to interpret *condensed* data using same context as *expanded* representations using SPARQL extensions such as in SciSPARQL [55] is another avenue of future work that could reduce graph size and complexity.

Independent testing is essential to strengthen the evidence presented in this work. We invite RDF developers to critically review the VCF Core vocabulary structure and conversion implementations, and VCF and ontology maintainers to review the *requirement* interpretations, *expected answers*, and model-comparison verdicts, in the collaborative style of community curation events [17, 56]. New fixtures, counterexamples, and disputed interpretations would directly improve both the vocabulary presented here and encourage greater VCF specification improvements that could have wide-ranging impacts on the field of genomics in general.

5. Conclusion

VCF Core provides a shared semantic foundation for representing VCF observations together with the context needed to interpret them. It combines reuse of existing models, explicit source structure, alternative sample representations and version-specific artifacts for VCF 4.1–4.5. All 104 constructs in the curated VCF 4.5 inventory are represented, and the claims that matter have been executed rather than asserted: the specification-derived checks were re-executed in full against a second producer, 693 of 840 passing for both with no divergence traceable to the vocabulary, all 109 records and 151 sample

genotypes round-trip through structured properties, and one integration example returns answers authored in advance together with their source provenance.

The remaining work is specific rather than general. Evidence is absent for 41 of the 94 *requirements*, concentrated in structural-variant fields, and thinner than we would like for allele-dependent values in particular; three structured accessors are not yet implemented in the cited converter; one QUAL datatype question is an interpretation the vocabulary deliberately leaves open; and the per-sample *condensed* profile cannot yet answer the hardest missingness question. None of these is a representation gap, and none is hypothetical — each is a named *requirement* with a recorded result.

What this foundation makes practical is the step from representing genomic files to connecting their contents with biomedical knowledge. Conversion engines can target the same model, integration layers can preserve the distinction between observations and interpretations, and SPARQL studies can examine medically relevant questions with traceable source evidence. The clinical scenarios in Section 4.6 remain future work: the VCF-side representation they would build on is in place and checked, while the biomedical models and reviewed instance mappings they equally require are not.

This study is intended to advertise a concrete resource (the VCF Core Vocabulary) that individuals and applications can reuse with a clearer account of what has been demonstrated.

Acknowledgments

Project funding provided from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).

Declaration on Generative AI

GPT-6 Astra (OpenAI) and Claude Opus 5 (Anthropic) assisted with the development of the VCF Core Vocabulary, the scripts used throughout this study, the initial writing of SPARQL queries used for testing, and the refinement of manuscript figures and text. The authors reviewed all AI outputs, all code generated by AI models, and all AI manuscript revisions to the manuscript. The authors take full responsibility for all content, tools, repositories, and paper contents.

References

- [1] C. Gaudet-Blavignac, J. L. Raisaro, V. Touré, S. Österle, K. Crameri, C. Lovis, A national, semantic-driven, three-pillar strategy to enable health data secondary usage interoperability for research within the Swiss Personalized Health Network: Methodological study, *JMIR Medical Informatics* 9 (2021) e27591. URL: <https://medinform.jmir.org/2021/6/e27591/>. doi:10.2196/27591.
- [2] T. E. Putman, K. Schaper, N. Matentzoglou, V. P. Rubinetti, F. S. Alquaddoomi, C. Cox, J. H. Caufield, G. Elsarboukh, S. Gehrke, H. Hegde, J. T. Reese, I. Braun, R. M. Bruskiewich, L. Cappelletti, S. Carbon, A. R. Caron, L. E. Chan, C. G. Chute, K. G. Cortes, V. De Souza, T. Fontana, N. L. Harris, E. L. Hartley, E. Hurwitz, J. O. B. Jacobsen, M. Krishnamurthy, B. J. Laraway, J. A. McLaughlin, J. A. McMurry, S. A. T. Moxon, K. R. Mullen, S. T. O’Neil, K. A. Shefchek, R. Stefancsik, S. Toro, N. A. Vasilevsky, R. L. Walls, P. L. Whetzel, D. Osumi-Sutherland, D. Smedley, P. N. Robinson, C. J. Mungall, M. A. Haendel, M. C. Munoz-Torres, The Monarch Initiative in 2024: an analytic platform integrating phenotypes, genes and diseases across species, *Nucleic Acids Research* 52 (2024) D938–D949. URL: <https://academic.oup.com/nar/article/52/D1/D938/7449493>. doi:10.1093/nar/gkad1082.
- [3] B. Smith, M. Ashburner, C. Rosse, J. Bard, W. Bug, W. Ceusters, L. J. Goldberg, K. Eilbeck, A. Ireland, C. J. Mungall, The OBI Consortium, N. Leontis, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, N. Shah, P. L. Whetzel, S. Lewis, The OBO Foundry: coordinated evolution of ontologies to support biomedical data integration, *Nature Biotechnology* 25 (2007) 1251–1255. URL: <https://www.nature.com/articles/nbt1346>. doi:10.1038/nbt1346.

- [4] F. Belleau, M.-A. Nolin, N. Tourigny, P. Rigault, J. Morissette, Bio2RDF: Towards a mashup to build bioinformatics knowledge systems, *Journal of Biomedical Informatics* 41 (2008) 706–716. URL: <https://www.sciencedirect.com/science/article/pii/S1532046408000415>. doi:10.1016/j.jbi.2008.03.004.
- [5] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, 1000 Genomes Project Analysis Group, The variant call format and VCFtools, *Bioinformatics* 27 (2011) 2156–2158. URL: <https://academic.oup.com/bioinformatics/article/27/15/2156/402296>. doi:10.1093/bioinformatics/btr330.
- [6] VCF, The Variant Call Format Specification: VCFv4.5 and BCFv2.2, 2026. URL: <https://samtools.github.io/hts-specs/VCFv4.5.pdf>, 25 February 2026 printing, version e821e4f; accessed 6 September 2026.
- [7] Global Alliance for Genomics and Health, A federated ecosystem for sharing genomic, clinical data, *Science* 352 (2016) 1278–1280. doi:10.1126/science.aaf6162.
- [8] Global Alliance for Genomics and Health, HTS Specifications: SAM/BAM and Related High-Throughput Sequencing File Formats, <https://github.com/samtools/hts-specs>, ??? Repository containing the Variant Call Format specifications, including VCF versions 4.1–4.5.
- [9] SAM/BAM Format Specification Working Group, HTS format specifications: VCF specifications and earlier versions, Official specification index, 2026. URL: <https://samtools.github.io/hts-specs/>, accessed 9 September 2026.
- [10] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. URL: <https://www.nature.com/articles/sdata201618>. doi:10.1038/sdata.2016.18.
- [11] C. Bahim, C. Casorrán-Amilburu, M. Dekkers, E. Herczog, N. Loozen, K. Repanas, K. Russell, S. Stall, The FAIR Data Maturity Model: An approach to harmonise FAIR assessments, *Data Science Journal* 19 (2020) 41. URL: <https://datascience.codata.org/articles/dsj-2020-041>. doi:10.5334/dsj-2020-041.
- [12] S. Decker, S. Melnik, F. van Harmelen, D. Fensel, M. Klein, J. Broekstra, M. Erdmann, I. Horrocks, The semantic web: the roles of XML and RDF, *IEEE Internet Computing* 4 (2000) 63–73. URL: <https://ieeexplore.ieee.org/document/877487/>. doi:10.1109/4236.877487.
- [13] C. Bizer, T. Heath, T. Berners-Lee, Linked Data—the story so far, *International Journal on Semantic Web and Information Systems* 5 (2009) 1–22. URL: <https://doi.org/10.4018/jswis.2009081901>. doi:10.4018/jswis.2009081901.
- [14] I. M. Lubin, N. Aziz, L. J. Babb, D. Ballinger, H. Bisht, D. M. Church, S. Cordes, K. Eilbeck, F. Hyland, L. Kalman, M. Landrum, E. R. Lockhart, D. Maglott, G. Marth, J. D. Pfeifer, H. L. Rehm, S. Roy, Z. Tezak, R. Truty, M. Ullman-Cullere, K. V. Voelkerding, E. A. Worthey, A. W. Zaranek, J. M. Zook, Principles and recommendations for standardizing the use of the next-generation sequencing variant file in clinical settings, *The Journal of Molecular Diagnostics* 19 (2017) 417–426. URL: <https://doi.org/10.1016/j.jmoldx.2016.12.001>. doi:10.1016/j.jmoldx.2016.12.001.
- [15] E. Garrison, Z. N. Kronenberg, E. T. Dawson, B. S. Pedersen, P. Prins, A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar, *PLOS Computational Biology* 18 (2022) e1009123. URL: <https://doi.org/10.1371/journal.pcbi.1009123>. doi:10.1371/journal.pcbi.1009123.

- [16] A. H. Wagner, L. Babb, G. Alterovitz, M. Baudis, M. Brush, D. L. Cameron, M. Cline, M. Griffith, O. L. Griffith, S. E. Hunt, D. Kreda, J. M. Lee, S. Li, J. Lopez, E. Moyer, T. Nelson, R. Y. Patel, K. Riehle, P. N. Robinson, S. Rynearson, H. Schuilenburg, K. Tsukanov, B. Walsh, M. Konopko, H. L. Rehm, A. D. Yates, R. R. Freimuth, R. K. Hart, The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification, *Cell Genomics* 1 (2021) 100027. doi:10.1016/j.xgen.2021.100027.
- [17] T. Katayama, S. Kawashima, G. Micklem, S. Kawano, J.-D. Kim, S. Kocbek, S. Okamoto, Y. Wang, H. Wu, A. Yamaguchi, et al., Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services, *F1000Research* 8 (2019) 1677. URL: <https://doi.org/10.12688/f1000research.18238.1>. doi:10.12688/f1000research.18238.1.
- [18] J. Baran, B. S. B. Durgahee, K. Eilbeck, E. Antezana, R. Hoehndorf, M. Dumontier, GFVO: the Genomic Feature and Variation Ontology, *PeerJ* 3 (2015) e933. URL: <https://peerj.com/articles/933/>. doi:10.7717/peerj.933.
- [19] J. T. Bolleman, C. J. Mungall, F. Strozzi, J. Baran, M. Dumontier, R. J. P. Bonnal, R. Buels, R. Hoehndorf, T. Fujisawa, T. Katayama, P. J. A. Cock, FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation, *Journal of Biomedical Semantics* 7 (2016) 39. URL: <https://link.springer.com/article/10.1186/s13326-016-0067-z>. doi:10.1186/s13326-016-0067-z.
- [20] E. D. S. Penha, E. Iriabho, A. Dussaq, D. M. de Oliveira, J. S. Almeida, Isomorphic semantic mapping of variant call format (VCF2RDF), *Bioinformatics* 33 (2017) 547–548. URL: <https://academic.oup.com/bioinformatics/article/33/4/547/2593587>. doi:10.1093/bioinformatics/btw652.
- [21] N. Mitsuhashi, L. Toyo-oka, T. Katayama, M. Kawashima, S. Kawashima, K. Miyazaki, T. Takagi, TogoVar: A comprehensive japanese genetic variation database, *Human Genome Variation* 9 (2022) 44. URL: <https://doi.org/10.1038/s41439-022-00222-9>. doi:10.1038/s41439-022-00222-9.
- [22] S. Kawashima, T. Fujisawa, T. Katayama, Genome variation ontology for annotation of complex structural variations, in: *Proceedings of the 14th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2023)*, volume 3415 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 126–127. URL: <https://ceur-ws.org/Vol-3415/paper-22.pdf>.
- [23] S. Prasanna, A. Kumar, D. Rao, E. J. Simoes, P. Rao, A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning, *Frontiers in Big Data* 7 (2025) 1466391. URL: <https://doi.org/10.3389/fdata.2024.1466391>. doi:10.3389/fdata.2024.1466391.
- [24] R. Janssen, UMC Utrecht Genetics, SPARQLing genomics, Version 0.99.11. Open-source software, 2020. URL: <https://gitlab.com/roelj/sparqling-genomics>, ontologies, RDF generators and a query interface for genomics; GPL-3.0. Released 19 June 2020.
- [25] A. Bodrug-Schepers, H. Chabane, G. Montoya, P. Serrano-Alvarado, R. Redon, A. Gaignard, Semantic beacons: a framework to support federated querying over genomic variants and public knowledge graphs, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 21–32. URL: https://ceur-ws.org/Vol-4196/paper_16.pdf.
- [26] M. Cazzaro, I. G. Gut, L. Menotti, M. Rueda, G. Silvello, HERO-Genomics: An ontology for integration and access of multicenter genomic data, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 33–42. URL: https://ceur-ws.org/Vol-4196/paper_17.pdf.
- [27] P. Larmande, B. Pittolat, N. Tando, Y. Pomie, B. G. Happi Happi, V. Guignon, M. Ruiz, AgroLD: a knowledge graph for the plant sciences, *BMC Genomic Data* 26 (2025) 73. URL: <https://doi.org/10.1186/s12863-025-01359-6>. doi:10.1186/s12863-025-01359-6.

- [28] E. Crum, R. Taelman, B. Buelens, G. Ertaylan, R. Verborgh, Semantifying genomic variant data: VCF to RDF conversion framework, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 199–200. URL: https://ceur-ws.org/Vol-4196/paper_6.pdf.
- [29] E. Crum, VCF Core Vocabulary, Version 2.1.2. GitHub repository, 2026. URL: <https://github.com/ecrum19/vcf-core-vocabulary/tree/3f06d18>, version 2.1.2 with the SSSOM alignment set; source snapshot of 14 September 2026.
- [30] World Wide Web Consortium, PROV-O: The PROV Ontology, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>.
- [31] K. Eilbeck, S. E. Lewis, C. J. Mungall, M. Yandell, L. Stein, R. Durbin, M. Ashburner, The sequence ontology: a tool for the unification of genome annotations, *Genome Biology* 6 (2005) R44. URL: <https://link.springer.com/article/10.1186/gb-2005-6-5-r44>. doi:10.1186/gb-2005-6-5-r44.
- [32] World Wide Web Consortium, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium, 2017. URL: <https://www.w3.org/TR/2017/REC-shacl-20170720/>.
- [33] E. Crum, Ontology Companion Generator (OCG), Version 1.4.0. Open-source software, 2026. URL: <https://github.com/ecrum19/ocg/tree/8963aa4>, MIT licence; released 1 September 2026. Documentation at <https://ecrum19.github.io/ocg/>.
- [34] A. Gómez-Pérez, Evaluation of ontologies, *International Journal of Intelligent Systems* 16 (2001) 391–409. doi:10.1002/1098-111X(200103)16:3<391::AID-INT1014>3.0.CO;2-2.
- [35] M. Sabou, M. Fernandez, Ontology (network) evaluation, in: M. C. Suárez-Figueroa, A. Gómez-Pérez, E. Motta, A. Gangemi (Eds.), *Ontology Engineering in a Networked World*, Springer, Berlin, Heidelberg, 2012, pp. 193–212. doi:10.1007/978-3-642-24794-1_9.
- [36] M. Grüninger, M. S. Fox, Methodology for the design and evaluation of ontologies, in: *Proceedings of the Workshop on Basic Ontological Issues in Knowledge Sharing, IJCAI-95*, Montreal, Canada, 1995. URL: <https://www.eil.utoronto.ca/wp-content/uploads/enterprise-modelling/papers/gruninger-ijcai95.pdf>.
- [37] M. C. Suárez-Figueroa, A. Gómez-Pérez, B. Villazón-Terrazas, How to write and use the ontology requirements specification document, in: *On the Move to Meaningful Internet Systems: OTM 2009*, volume 5871 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2009, pp. 966–982. doi:10.1007/978-3-642-05151-7_16.
- [38] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, R. García-Castro, LOT: An industrial oriented ontology engineering framework, *Engineering Applications of Artificial Intelligence* 111 (2022) 104755. doi:10.1016/j.engappai.2022.104755.
- [39] S. Peroni, A simplified agile methodology for ontology development, in: *OWL: Experiences and Directions – Reasoner Evaluation*, volume 10161 of *Lecture Notes in Computer Science*, Springer, Cham, 2017, pp. 55–69. doi:10.1007/978-3-319-54627-8_5.
- [40] World Wide Web Consortium, SPARQL 1.1 Query Language, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>.
- [41] E. Blomqvist, A. Seil Sepour, V. Presutti, Ontology testing: Methodology and tool, in: *Knowledge Engineering and Knowledge Management*, volume 7603 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2012, pp. 216–226. doi:10.1007/978-3-642-33876-2_20.
- [42] D. Wiśniewski, J. Potoniec, A. Ławrynowicz, C. M. Keet, Analysis of ontology competency questions and their formalizations in SPARQL-OWL, *Journal of Web Semantics* 59 (2019) 100534. doi:10.1016/j.websem.2019.100534.
- [43] VCF-RDFizer maintainers, VCF-RDFizer, Version 3.0.3. Open-source software, 2026. URL: <https://github.com/ecrum19/VCF-RDFizer/tree/be658a2>, tag v3.0.3; converted and validated 14 September 2026.

- [44] N. Matentzoglou, J. P. Ballhoff, S. M. Bello, C. Bizon, M. Brush, T. J. Callahan, C. G. Chute, W. D. Duncan, C. T. Evelo, D. Gabriel, J. Graybeal, A. Gray, B. M. Gyori, M. Haendel, H. Harmse, N. L. Harris, I. Harrow, H. B. Hegde, A. L. Hoyt, C. T. Hoyt, D. Jiao, E. Jiménez-Ruiz, S. Jupp, H. Kim, S. Koehler, T. Liener, Q. Long, J. Malone, J. A. McLaughlin, J. A. McMurtry, S. Moxon, M. C. Munoz-Torres, D. Osumi-Sutherland, J. A. Overton, B. Peters, T. Putman, N. Queralt-Rosinach, K. Shefchek, H. Solbrig, A. Thessen, T. Tudorache, N. Vasilevsky, A. H. Wagner, C. J. Mungall, A simple standard for sharing ontological mappings (SSSOM), Database 2022 (2022) baac035. URL: <https://academic.oup.com/database/article/doi/10.1093/database/baac035/6591806>. doi:10.1093/database/baac035.
- [45] S. Bishop, M. Fairbairn, M. Norrish, P. Sewell, M. Smith, K. Wansbrough, Rigorous specification and conformance testing techniques for network protocols, as applied to TCP, UDP, and sockets, in: Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM), ACM, 2005, pp. 265–276. URL: <https://doi.org/10.1145/1080091.1080123>. doi:10.1145/1080091.1080123.
- [46] Y. N. Niu, E. G. Roberts, D. Denisko, M. M. Hoffman, Assessing and assuring interoperability of a genomics file format, Bioinformatics 38 (2022) 3327–3336. URL: <https://doi.org/10.1093/bioinformatics/btac327>. doi:10.1093/bioinformatics/btac327.
- [47] S. Beier, A. Fiebig, C. Pommier, I. Liyanage, M. Lange, P. J. Kersey, S. Weise, R. Finkers, B. Koylass, T. Cezard, M. Courtot, B. Contreras-Moreira, G. Naamati, S. Dyer, U. Scholz, Recommendations for the formatting of variant call format (VCF) files to make plant genotyping data FAIR, F1000Research 11 (2022) 231. URL: <https://doi.org/10.12688/f1000research.109080.2>. doi:10.12688/f1000research.109080.2, version 2; peer review: 2 approved.
- [48] Med2RDF contributors, Med2RDF ontology, GitHub repository, n.d. URL: <https://github.com/med2rdf/med2rdf-ontology>, accessed 8 September 2026.
- [49] gvar, Genomic variants schema (gvar), GitHub repository, n.d. URL: <https://github.com/swat4hcls-2025-genomic-variation/genomic-variant-schema>, accessed 6 September 2026.
- [50] World Wide Web Consortium, SKOS Simple Knowledge Organization System Reference, W3C Recommendation, World Wide Web Consortium, 2009. URL: <https://www.w3.org/TR/skos-reference/>.
- [51] C. R. Lee, J. A. Luzum, K. Sangkuhl, R. S. Gammal, M. S. Sabatine, C. M. Stein, D. F. Kisor, N. A. Limdi, Y. M. Lee, S. A. Scott, J.-S. Hulot, D. M. Roden, A. Gaedigk, K. E. Caudle, T. E. Klein, J. A. Johnson, A. R. Shuldiner, Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update, Clinical Pharmacology & Therapeutics 112 (2022) 959–967. doi:10.1002/cpt.2526.
- [52] PharmCAT contributors, PharmCAT methods, Official documentation, 2026. URL: <https://pharmcat.org/methods/>, accessed 9 September 2026.
- [53] Human Phenotype Ontology Consortium, The human phenotype ontology, Project website and documentation, 2026. URL: <https://hpo.jax.org/>, accessed 9 September 2026.
- [54] M. Griffith, N. C. Spies, K. Krysiak, J. F. McMichael, A. C. Coffman, A. M. Danos, B. J. Ainscough, C. A. Ramirez, D. T. Rieke, L. Kujan, E. K. Barnell, A. H. Wagner, Z. L. Skidmore, A. Wollam, C. J. Liu, M. R. Jones, R. L. Bilski, R. Lesurf, Y.-Y. Feng, N. M. Shah, M. Bonakdar, L. Trani, M. Matlock, A. Ramu, K. M. Campbell, G. C. Spies, A. P. Graubert, K. Gangavarapu, J. M. Eldred, D. E. Larson, J. R. Walker, B. M. Good, C. Wu, A. I. Su, R. Dienstmann, A. A. Margolin, D. Tamborero, N. Lopez-Bigas, S. J. M. Jones, R. Bose, D. H. Spencer, L. D. Wartman, R. K. Wilson, E. R. Mardis, O. L. Griffith, CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer, Nature Genetics 49 (2017) 170–174. doi:10.1038/ng.3774.
- [55] A. Andrejev, T. Risch, Scientific SPARQL: Semantic web queries over scientific data, in: 2012 IEEE 28th International Conference on Data Engineering Workshops (ICDEW), IEEE, 2012, pp. 5–10. URL: <https://doi.org/10.1109/ICDEW.2012.67>. doi:10.1109/ICDEW.2012.67.

- [56] SWAT4HCLS 2025 BioHackathon genomic variation participants, Genomic variant RDF representations: mapping declarations to GA4GH VRS, Project site and report, 2025. URL: <https://swat4hcls-2025-genomic-variation.github.io/genomic-variant-schema/>, declarations transcribed as SSSOM sets and republished with the VCF Core Vocabulary; accessed 8 September 2026.